Tower Siege

A tower-defense game built on C++ data structures & algorithms

UD
UNIVERSIDAD DISTRITAL
Francisco José de Caldas

Daniel Santiago Pérez Madera — Code 20231020203
Computer Science I · Season 2026-I
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia

Introduction

Data structures and algorithms are hard to grasp from textbooks alone. Tower Siege turns them into a **playable tower-defense game**, where the player places towers to stop enemies reaching the castle.

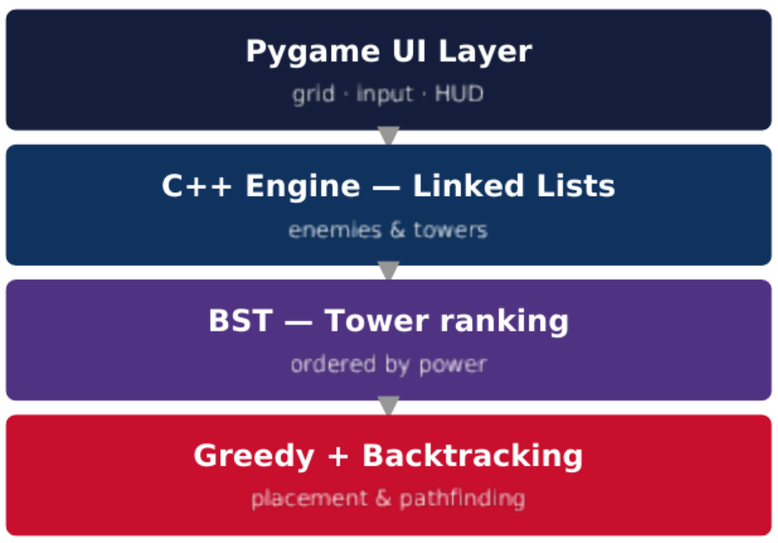
The two main challenges were managing **raw pointers** in C++ and making two algorithms — **Greedy** and **Backtracking** — cooperate in the same game loop.

Goal

> **Question:** Can a tower-defense game be built purely with hand-made C++ data structures to show Greedy and Backtracking in action?
>
> **Product:** A desktop game with a C++ engine and a Pygame interface.

Proposed Solution

A **C++ engine** uses **linked lists** for enemies and towers, and a **BST** to rank towers by power — no external libraries. A **Pygame** layer renders the grid and HUD.



The Game in Action

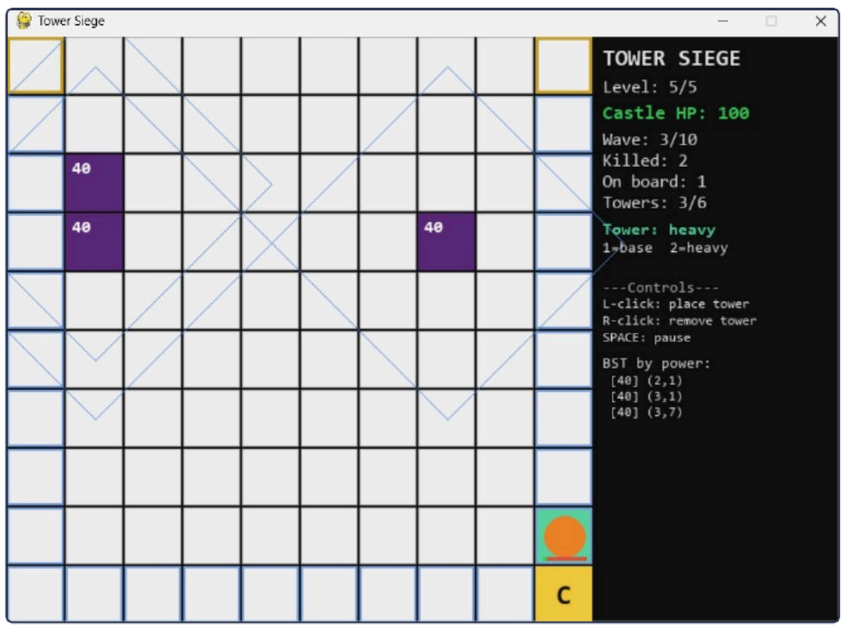


Fig 1. Active wave — towers (purple), enemy spawn and live BST ranking in the HUD.

Experiments

Each data structure was unit-tested, and the full game loop was checked with integration tests for pointer integrity and algorithm output.

Test	Result
Linked list insert / remove	**PASS**
BST insert by power	**PASS**
Greedy tower suggestion	**PASS**
Backtracking pathfinding	**PASS**
Greedy + Backtracking combined	**FREEZE**

Results

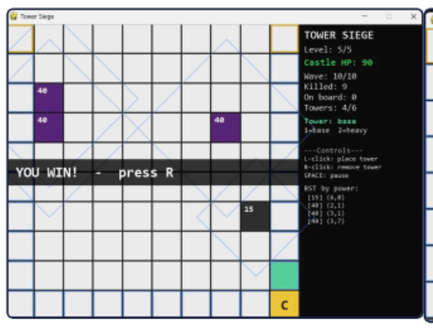


Fig 2. Level cleared.

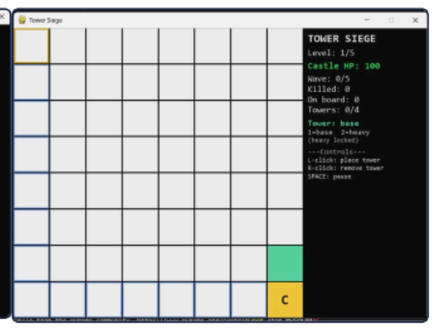


Fig 3. Towers firing.

Strengths: the game runs fully on hand-built structures; all five levels are playable; both algorithms work correctly **in isolation**.

Key finding: when **Greedy** places towers around the castle it can **block every route**. **Backtracking** then searches endlessly and the game **freezes**.

Conclusions

A tower-defense game **can** be built purely on custom C++ data structures, making Greedy and Backtracking tangible.

The freeze shows a key lesson: **two locally-correct algorithms can conflict globally**. Future work: stop Greedy from sealing every path, and limit Backtracking's search depth.

References

[1] Pygame Documentation. pygame.org/docs

[2] N. Lohmann, *JSON for Modern C++*. github.com/nlohmann/json

[3] Python Software Foundation, *json module*. docs.python.org/3/library/json.html